이권희 포트폴리오

010-4795-8032 · lkh8033@gmail.com · github.com/kwonhee1 · velog.io/@lkh8033

문제 해결을 좋아하는 성향이 개발로 이어졌습니다. 다양한 문제들을 만나면서, 저만의 해결책을 코드로 풀어내는 과정이 즐거웠습니다. 정답이 정해져 있지 않은 코딩의 자유도가 그 매력을 더했습니다.

[프로젝트 소개]



Arubi AI 여행일정 생성 앱

팀 프로젝트

2026.03 ~ 2026.12

Backend

최소한의 입력으로 사용자에게 맞는 여행 일정을 생성하는 AI 여행 플래너

Spring Boot · MySQL · Docker · FastAPI



HooYah 요트 통합 관리 앱

개인 프로젝트

2026.01 ~ 2026.03

Backend 1 인

요트 부품별 정비 이력과 다음 정비 시점을 관리하는 앱을 MSA 로 재구성

Spring Boot · Netty · Redis

WithU 크라우드 펀딩 플랫폼

팀 프로젝트

2025.04 ~ 2025.11

Backend 1 인

누구나 프로젝트를 열고 후원받을 수 있는 크라우드 펀딩 플랫폼

Spring Boot · JPA · Spring Security · MySQL

Arubi (AI 여행일정 생성 앱) GitHub

팀 프로젝트 (2026.03 ~ 2026.12) · Backend 2 · Frontend 2 · 4 학년 졸업작품

사용 기술 Spring Boot · MySQL · Docker · FastAPI · draw.io

[프로젝트 설명]

지역 · 날짜 · 인원만 입력하면 AI 가 검증된 장소 데이터를 바탕으로 동선과 시간까지 최적화한 여행 일정을 자동 생성하는 여행 플래너, ARUBI 입니다. 생성된 일정은 지도에서 동선을 확인하며 직접 편집할 수 있고, 여행 중에는 실시간 위치와 환율 · 날씨 정보를 함께 제공합니다.

단순히 AI 가 일정을 생성해주는 것을 넘어 사용자에 맞는 장소 추천, 실시간 일정 수정, 여행 도구 등을 가진 여행 플래너입니다.

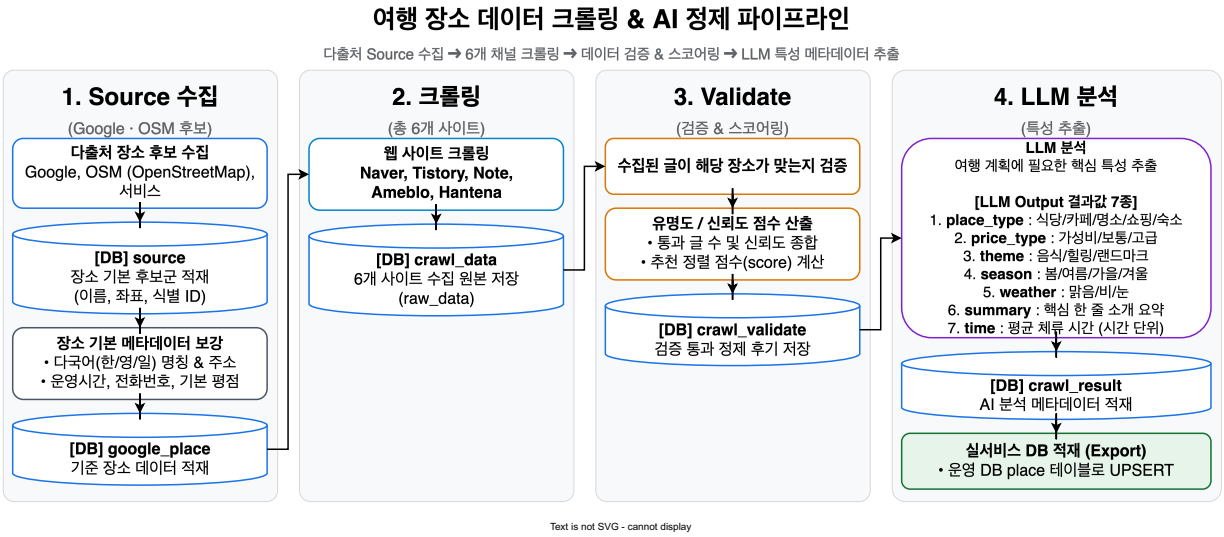
[주요 구현 내용]

Data 파이프라인 장소 DB 파이프라인 Velog

지역 단위로 장소 목록을 내려주는 API 가 없어, 장소 DB 를 직접 구성하는 크롤링 파이프라인을 구현했습니다.

Source → Google API 기본 정보 조회 → 크롤링 → 검증 → LLM 분석의 단계로 분리하여 구현하였습니다.

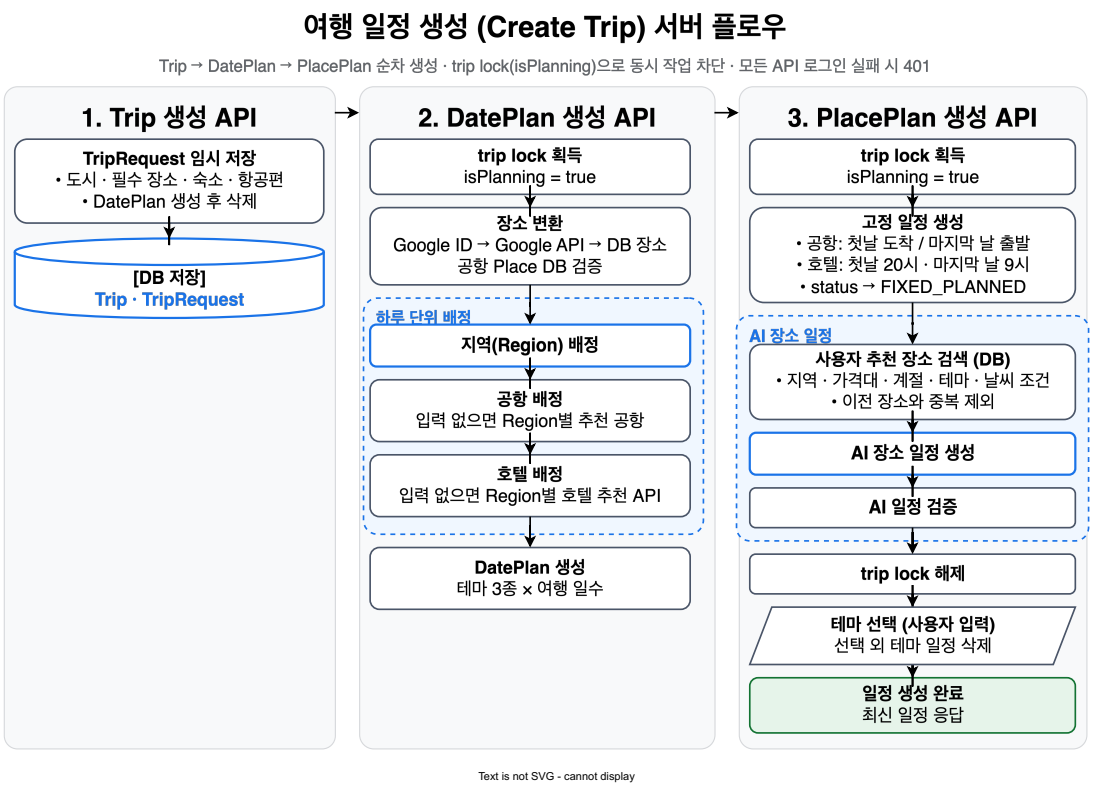
Google, Naver 에서는 제공하지 않는 장소별 테마 · 가격대 · 추천 계절 · 평균 소요 시간 등을 추출하는 것이 목표입니다.



AI 일정 생성 AI 일정 생성 기능 구현 Velog

필수 정보 (날짜, 지역, 인원)을 입력하면 장소 일정과 이동 정보를 생성하는 기능입니다.

필수 정보들을 Trip 에 저장하고, 일정을 날짜별로 분리해서 DatePlan 을 만들고 필수 장소들을 배정합니다. DB 에서 1 차 검증된 장소들을 선별하여 AI 가 장소 일정을 만듭니다. 이후 장소 간의 이동 정보는 API 를 사용하여 채워집니다.

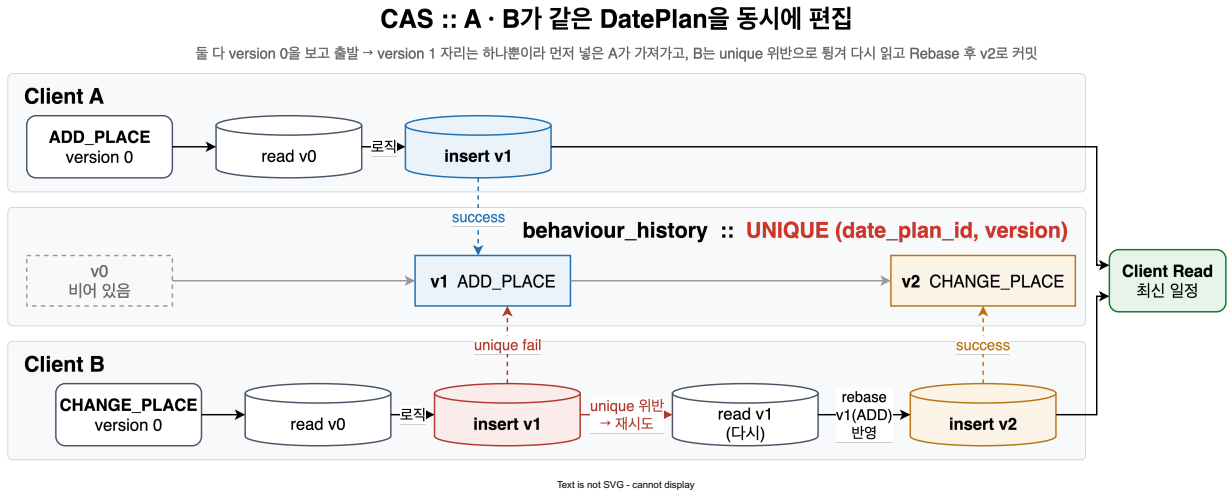


Text is not SVG - cannot display

일정 수정 (동시성) CAS 동시 수정 구현 Velog

여러 명의 동행자가 같은 일정을 동시에 수정할 수 있는 기능을 구현했습니다.

Lock 을 잡지 않는 CAS(Compare and Swap) 방식으로 동시성을 처리했습니다. Client 가 편집 요청(Behaviour)과 자신의 version 을 보내면, 그 version 이후 DB 에 쌓인 Behaviour 들을 읽어 요청에 반영(rebase)한 뒤 저장합니다. 다른 요청이 먼저 선점해 실패하면 이전의 rebase 과정을 반복합니다.



Text is not SVG - cannot display

HooYah (요트 통합 관리 앱) GitHub

개인 프로젝트 (2026.01 ~ 2026.03) · Backend 1인 · 고용노동부 일경험 프로젝트

사용 기술 Spring Boot · Netty · Redis · Docker · K6

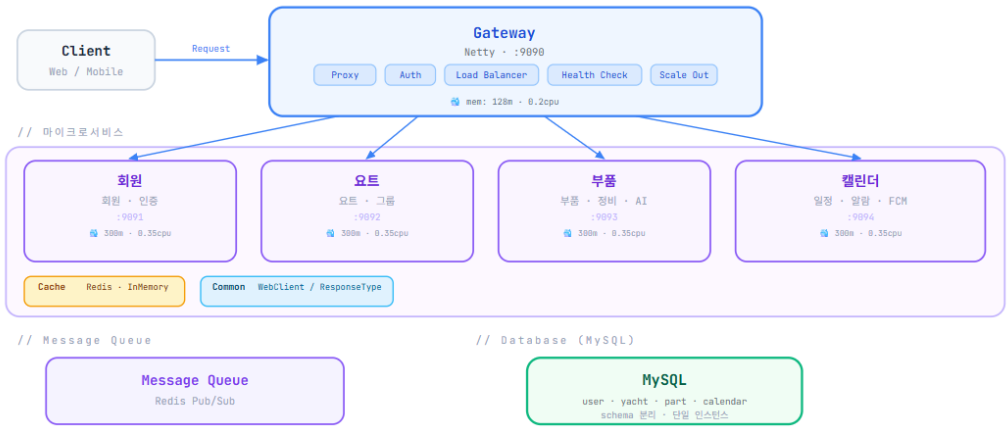
배포 Raspberry Pi 4 · Docker

[프로젝트 설명]

HooYah 는 요트를 등록하고 부품별 정비 이력과 다음 정비 시점을 관리하는 요트 통합 관리 앱입니다. 부품마다 정비 주기가 달라 수기 관리가 어렵다는 문제를 해결하기 위한 앱입니다. 사용자가 요트와 부품과 정비 이력을 기록하면 다음 정비 일정이 캘린더에 자동으로 추가됩니다.

팀 프로젝트로 시작한 프로젝트에서 모노리스 완성 이후 혼자서 MSA 환경으로 변경하면서 시작한 프로젝트입니다 하나로 결합돼 있던 서비스 로직들을 도메인 단위로 분리하고 MSA 아키텍쳐를 적용하는 것이 목표였습니다. User · Yacht · Part · Calendar 네 개의 MicroService 와 Gateway 로 나누고 MQ 를 도입하여, Raspberry Pi 4(RAM 4Gi) 제한 환경에 MSA 아키텍쳐를 배포했습니다.

[아키텍처]



[주요 구현 내용]

Gateway README

Proxy · Auth · Load Balance · Health Check · Auto Scale Out 을 담당하는 Service 입니다.

Netty 라이브러리를 사용한 저수준의 Proxy Server 와 주기적으로 각 MicroService 의 상태를 수집하여 Load Balancer 와 Scale Out 담당합니다.

Cache README

MicroService 에서 공통으로 사용하기 위한 Cache 라이브러리 입니다.

서비스 중에 Redis Container 가 죽으면 InMemory 로 자동 전환되어 작동합니다.

도메인별 MicroService 분리

한개의 모놀리식 서비스를 Domain 별로 User · Yacht · Part · Calendar 네 개의 MicroService 로 분리했습니다. Domain 간 FK 를 제거해 스키마를 완전히 분리하고, 한 개의 service 에 섞여 있던 로직들 또한 Domain 별로 분리하였습니다.

[트러블 슈팅]

MicroService Container 가 죽는 원인 찾기 GitHub Issue #3

Situation 성능 테스트 도중 MicroService Container 가 반복해서 죽어서, Gateway 에서 MicroService 를 보호하는 기능을 구현하고 싶었습니다. 기능 구현 앞서 Container 의 생존을 보장하려면 `Request 개수가 많아지면 죽는다`가 아니라 **Container 가 죽는 정확한 원인**을 먼저 찾아야 했습니다.

Action

1. Docker OOM Killer 가 Container 를 종료시킨 것을 확인 (Docker exit code, System log)
2. jshell 로 JVM 이 인식하는 메모리 limit 확인 → Docker limit 제한과 다른 값을 잡고 있었음
3. JVM UseContainerSupport 옵션이 작동하지 않는 것을 확인

Result eclipse-temurin:21-jdk 에서 JVM UseContainerSupport Option 이 적용되고 있지 않다는 것을 확인했습니다. JDK Docker 이미지를 amazoncorretto:21-jdk 로 교체해 Container 메모리 제한을 인식하도록 했습니다.

Request 개수가 많아져도 서버가 느려질 뿐 죽지 않게 되었고, 이어서 Gateway 가 MicroService 별로 정해진 Request 만 허용해 각 서비스의 응답 속도를 보장하는 기능을 구현할 수 있었습니다.

회원가입 API 동시성 문제 해결 GitHub Issue #10

Situation 성능 Test 를 위해 회원가입 API 를 동시에 여러 번 호출하니 같은 email 을 가진 User 가 중복 생성되는 문제를 발견했습니다.

중복 email 을 가진 User 2 개 모두 Error 가 발생하여 모든 기능을 사용할 수 없었습니다. Transaction 이 있어도 동시성이 발생할 수 있다는 사실을 배우게 되었습니다.

Action

1. DB email unique 제약 추가 + 비관적 Lock 적용
2. 오래 걸리는 PasswordEncoder 를 Transaction 밖으로 분리 (TransactionTemplate)

Result Multi Thread 환경에서 같은 email 로 동시에 가입을 요청하는 Test 를 작성해 중복 User 가 생성되지 않는 것을 확인했습니다.

Lock 과정에서 오래 걸리는 PasswordEncoder 를 Transaction 밖으로 분리하면서 DB Connection 점유를 최소화하였습니다.

WithU (크라우드 펀딩 플랫폼) GitHub

팀 프로젝트 (2025.04 ~ 2025.11) · Backend 1 · Frontend 3 · 2025 동양미래대학교 졸업작품

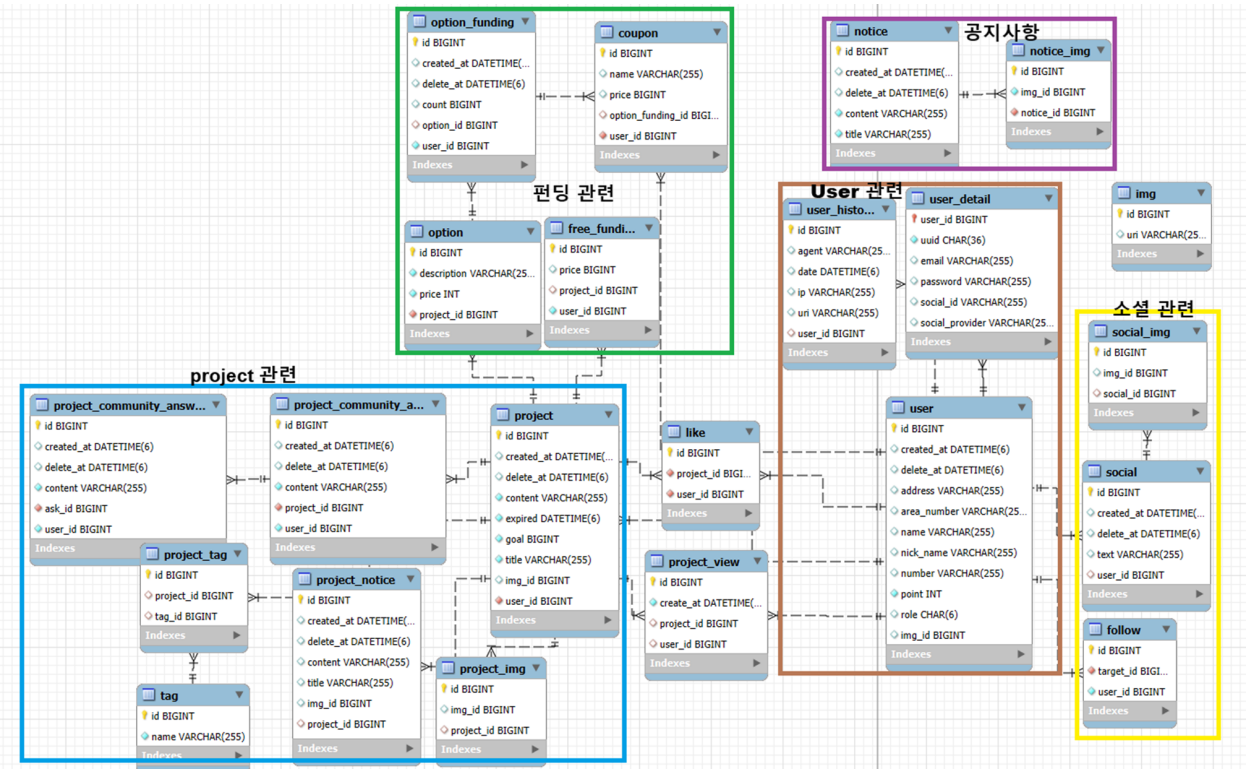
사용 기술 Java 21 · Spring Boot · JPA · Spring Batch · Spring Security · JWT · OAuth2 · MySQL

배포 Raspberry Pi 4 · Docker · GitHub CI/CD · Nginx

[프로젝트 설명]

WithU 는 아이디어를 가진 누구나 프로젝트를 열고 후원받을 수 있는 크라우드 펀딩 플랫폼입니다. 창작자가 목표 금액과 기간을 정해 펀딩 프로젝트를 등록하면 다양한 사람들로부터 후원을 받아 프로젝트를 진행할 수 있습니다.

[ERD]



[주요 구현 내용]

Toss 결제 API

Toss 결제를 연동하여 결제 API 구현

[트러블 슈팅]

Project List 조회 API [API 확장 / 성능 최적화 Velog]

Situation Project List 는 펀딩 Project 의 주요 정보를 한눈에 보여 주는 요약 목록 API 로, 서비스에서 가장 많이 쓰이는 API 였습니다. 그만큼 호출되는 곳도 많고, 한 번에 보여 줘야 하는 정보도 많았습니다.

조회 조건	내가 생성한 · 내가 펀딩한 · 좋아요한 · 최근 조회한	4 가지 조건
공통 필터	검색어 · 태그 · 프로젝트 상태	3 가지 필터
정렬 조건	추천 · 달성률 · 펀딩 인원 · 생성일 · 마감일 · 조회수	6 가지 정렬
연관 테이블	한 번 조회에 이미지 · 태그 · 좋아요 · 조회수 등	9 개 테이블

조건마다 함수의 파라미터를 늘리는 방식으로는, 여러가지의 조건을 모두 처리하기 어려웠습니다.
Table 이 늘어날수록 코드가 많아지고 유지 보수하기 어려워졌습니다.

Action

1. QueryDSL 도입
2. 각 API 마다 필요한 조건을 먼저 받아 where 절을 완성할 Builder Class 를 구현
3. Sub Query → Left Join 변경

Result 한 개의 함수로 여러 조건의 목록 조회를 처리할 수 있게 되었고, API 에서는 필요한 조건만 넘기면 됩니다.
Sub Query 를 Join 으로 변경하면서 조회 시간도 함께 줄었습니다.

> 내가 생성한 project 조회 946ms → **21ms**
>
> Funding Data 포함 조회 1,118ms → **344ms**

Img Transaction 구현 [서버에 이미지 파일 저장하기 Velog]

Situation 이미지를 파일을 서버에 직접 저장하는 방법으로 구현했을 때 발생한 문제입니다. DB 는 Spring Transaction 으로 RollBack 되어도 서버에 저장된 이미지 파일은 함께 RollBack 되지 않았습니다.

Action Spring AOP 를 사용해 이미지 RollBack 을 위한 ImgTransaction 을 구현했습니다.
Img 파일을 다루는 함수 전에 Aop 를 설정하고, Aop 가 끝날 때 성공 여부에 따라 이미지 파일을 삭제합니다.

Success (정상 종료) : deleteImgs 의 이미지 파일을 삭제

RollBack (실패) : savedImgs 의 이미지 파일을 삭제

Result 이미지를 사용하는 Service 함수에 @ImgTransaction 어노테이션만 붙이면, RollBack 이 발생할 때 이미지 파일도 함께 RollBack 됩니다.

AuthorizationManager 커스터마이징 [권한별 인가 실패 응답 세분화 Velog]

Situation 결제 · 펀딩 API 는 주소, 전화번호 등의 정보를 모두 입력한 사용자만 호출할 수 있습니다. 소셜 로그인으로 가입한 SOCIAL 권한 사용자는 이 정보가 채워지지 않은 상태이므로 결제 · 펀딩 API 접근을 금지했습니다. 이때 결제 · 펀딩 API 는 거절 원인을 프론트에 반환해야 했습니다.

Action Uri 마다 권한을 판단하는 AuthorizationManager 를 커스터마이징 하여, 인가 실패시 실패의 원인을 front 에 전달하게 했습니다.

1. AuthorizationManager 를 커스터마이징 실패 원인을 분류하고, 알맞은 ErrorCode
2. AccessDeniedHandler 에서 ErrorCode 를 꺼내 프론트에 거절 원인을 전달

Result 결제 · 펀딩 API 호출 시 권한별로 다음과 같이 응답합니다.

	비로그인	SOCIAL	USER
결제 · 펀딩 API	불가	불가	가능
실패 원인	No authentication please login	Additional data required	—
Http Code	401	403	—